Akademia WSB

Rok akademicki 2026/2027
Przedmiot Bezpieczeństwo aplikacji internetowych
Forma zajęć Laboratorium
Prowadzący zajęcia dr inż. Jarosław Homa
Nr grupy 7

Nazwisko i imię	Nr indeksu
Kacper Bernaś	**74081**

Streszczenie

Celem projektu było przygotowanie lokalnej aplikacji demonstracyjnej, która rejestruje istotne zdarzenia bezpieczeństwa, wiąże je z konkretnym żądaniem HTTP oraz prezentuje zdarzenia i wykryte alerty w panelu administracyjnym. Projekt koncentruje się na obserwowalności bezpieczeństwa: nie tylko na zapisaniu informacji o zdarzeniu, lecz także na umożliwieniu administratorowi zauważenia powtarzających się prób logowania i prób dostępu do chronionych zasobów.

Aplikację zbudowano w Pythonie z wykorzystaniem Flask, SQLite, szablonów HTML/Jinja i Docker Compose. Zaimplementowano logowanie użytkowników, kontrolę dostępu opartą na rolach,

rejestrowanie zdarzeń, identyfikator korelacyjny request_id, filtrowanie tabeli zdarzeń oraz reguły generujące alerty dla wybranych wzorców zachowania. Testy wykonano na syntetycznych kontach w lokalnym środowisku.

Raport opisuje działanie rozwiązania, model zdarzeń, scenariusze testowe, uzyskane wyniki i ograniczenia.

Cel i zakres projektu

Projekt realizuje temat P9 „Monitoring i obserwowalność bezpieczeństwa". Głównym celem jest pokazanie, w jaki sposób aplikacja internetowa może rejestrować zdarzenia istotne z punktu widzenia bezpieczeństwa, wykrywać wybrane wzorce podejrzanej aktywności i udostępniać wyniki w jednym miejscu.

Zakres obejmuje lokalną aplikację demonstracyjną. Testowane są zdarzenia związane z uwierzytelnianiem, wylogowaniem, odmową dostępu, zmianą roli oraz utworzeniem konta. W zakresie projektu nie znajduje się pełny system SIEM, integracja z zewnętrznym dostawcą logów ani wdrożenie produkcyjne.

Cele szczegółowe

Przyjęto następujące cele: ustalenie, kto wykonał daną czynność tzw. User accounting, kiedy do niej doszło i czego dotyczyła. Zapis wyniku operacji i kontekstu żądania, korelacja zdarzeń za pomocą identyfikatora request_id. Wykrywanie wybranych serii nieudanych logowań i odmów dostępu. Ograniczenie dostępu do funkcji administracyjnych oraz prezentacja wyników na dashboardzie.

Zakres i granice

Aplikacja działa w kontrolowanym środowisku lokalnym, uruchamianym przez Docker Compose. Używane konta i dane są testowe. Wyniki nie stanowią oceny bezpieczeństwa produkcyjnej aplikacji ani kompletnego audytu zgodności. Zaimplementowane reguły korelacji są demonstracyjne i wykorzystują określone okna czasowe oraz progi.

Środowisko i uruchomienie

Projekt jest przeznaczony do uruchomienia lokalnie. Wykorzystuje Docker Desktop, Docker Compose, obraz bazujący na python:3.13-slim, Flask oraz SQLite. Lokalna instalacja Pythona w systemie Windows nie jest wymagana do uruchomienia aplikacji, ponieważ środowisko aplikacyjne działa w kontenerze.

Przed uruchomieniem należy mieć zainstalowany i uruchomiony Docker Desktop oraz sklonowane lub rozpakowane repozytorium projektu. W katalogu głównym należy utworzyć plik .env z losowym SECRET_KEY. W projekcie plik .env jest wykluczony przez .gitignore. Konfiguracja wymaga ustawienia SECRET_KEY, a aplikacja nie powinna uruchamiać się bez tej wartości.

Przykładowa procedura uruchomienia w PowerShell:

```
cd C:\projekty\p9-monitoring
 docker compose up –d --build
 docker compose ps
```

Po uruchomieniu aplikacja jest dostępna pod adresem http://localhost:5000. Do demonstracji należy używać wyłącznie syntetycznych kont i danych testowych. Dane logowania testowych użytkowników:
Student | StudentLab2026!
Admin | AdminLab2026!

Repozytorium projektu

Kod źródłowy projektu „Monitoring i obserwowalność bezpieczeństwa” jest dostępny w repozytorium GitHub:

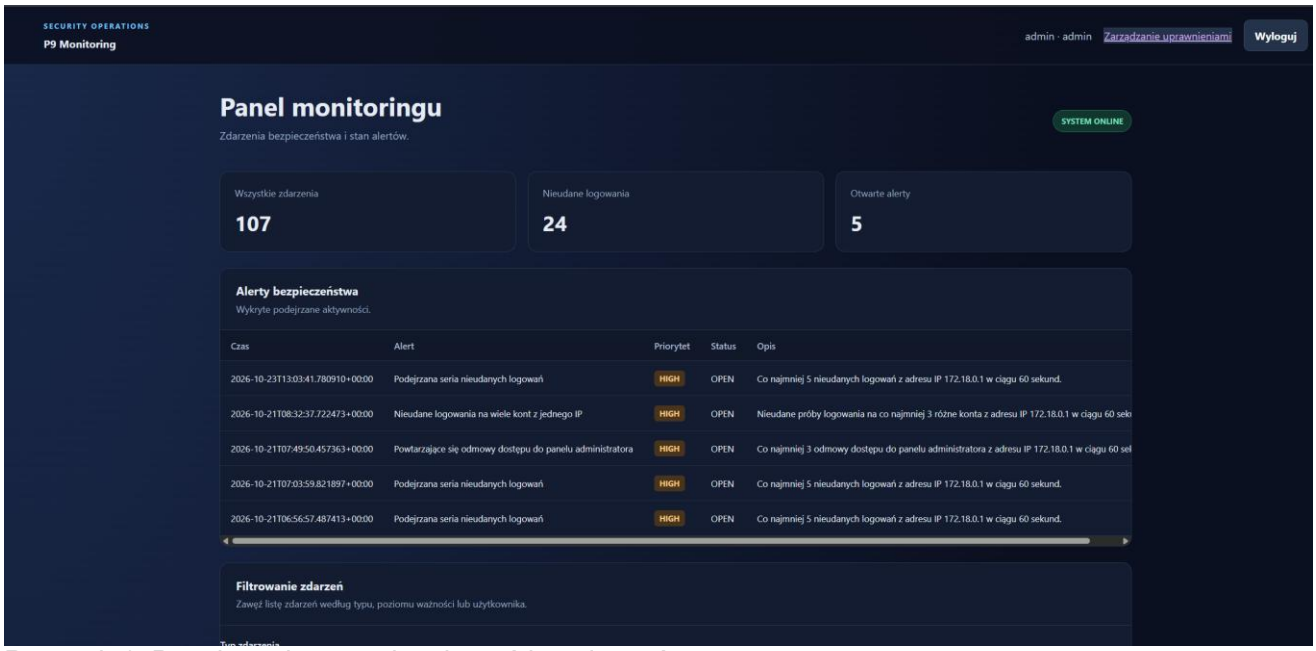
https://github.com/KacperBernas/p9-monitoring

Repozytorium zawiera kod aplikacji Flask, konfigurację środowiska Docker Compose oraz pliki niezbędne do uruchomienia i przetestowania systemu monitorowania zdarzeń bezpieczeństwa.

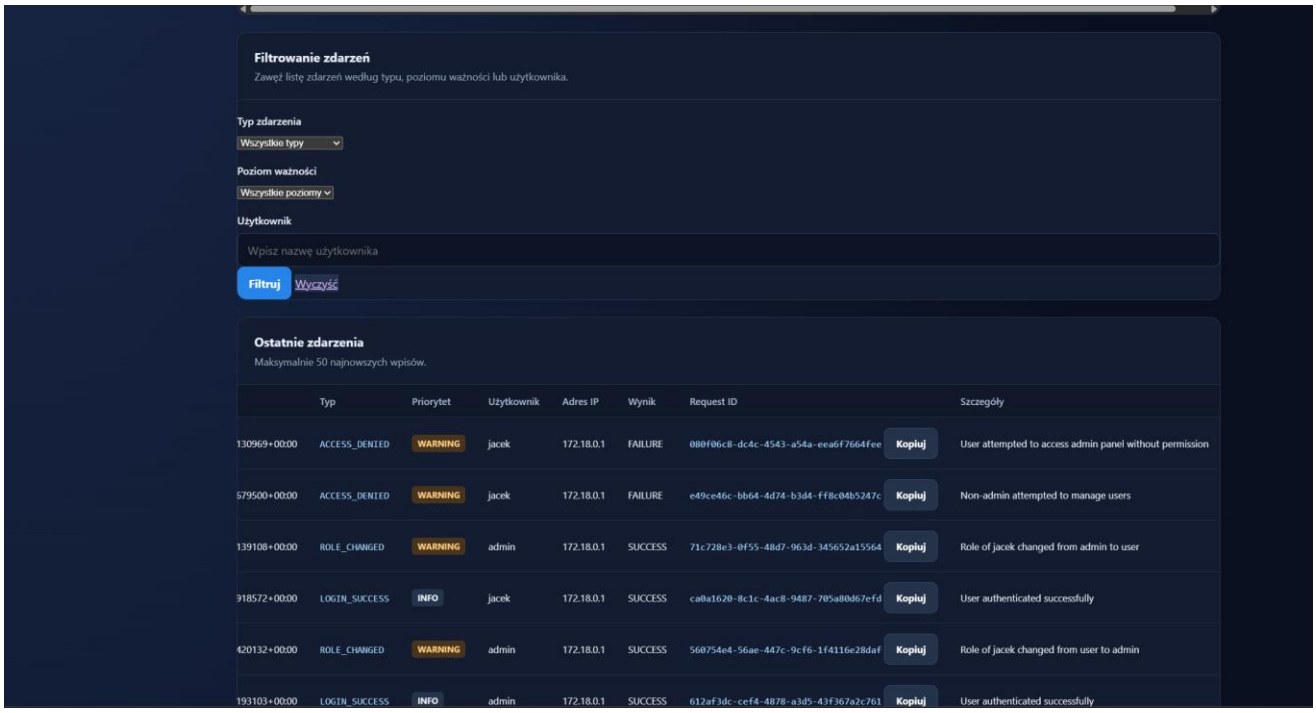
Architektura rozwiązania

Rozwiązanie składa się z aplikacji Flask, bazy SQLite oraz interfejsu HTML renderowanego po stronie serwera. Kontenery i konfigurację uruchomienia obsługuje Docker Compose. Dane zdarzeń są zapisywane w bazie, a dashboard pobiera je i prezentuje administratorowi. Logika rejestrowania zdarzeń oraz reguły korelacji są wydzielone do modułów aplikacji.

Element	Rola w projekcie
Flask / app.py	Obsługa żądań HTTP, sesji, logowania, kontroli dostępu i routingu.
SQLite / app/database.py	Przechowywanie użytkowników, zdarzeń bezpieczeństwa i danych alertów.
app/security_events.py	Zapisywanie zdarzeń bezpieczeństwa i uruchamianie właściwej korelacji.
app/correlation.py	Ocena zdarzeń w oknach czasowych i tworzenie alertów po przekroczeniu progów.
Szablony HTML/Jinja	Widoki logowania, dashboardu i zarządzania kontami oraz rolami.
Docker Compose	Powtarzalne uruchomienie aplikacji i przekazanie konfiguracji środowiskowej.



Rysunek 1. Panel monitorowania zdarzeń bezpieczeństwa.



Rysunek 2. Panel monitorowania zdarzeń bezpieczeństwa nr 2.

Model zdarzeń i rejestrowanie

Aplikacja zapisuje zdarzenia bezpieczeństwa w ustrukturyzowanej postaci. Rekord zawiera między innymi czas wystąpienia, typ zdarzenia, poziom ważności, nazwę użytkownika, adres IP, endpoint, wynik operacji, opis oraz request_id. Dzięki temu zdarzenia można filtrować i porównywać, zamiast analizować wyłącznie swobodny tekst.

Pole	Znaczenie
timestamp	Data i czas wystąpienia zdarzenia.
event_type	Rodzaj zdarzenia, np. LOGIN_FAILED lub

	ROLE_CHANGED.
severity	Poziom ważności przypisany do zdarzenia.
username	Konto, którego dotyczy zdarzenie dla części zdarzeń może nie być znane.
ip_address	Adres IP źródła żądania widziany przez aplikację.
endpoint	Ścieżka lub obszar aplikacji, którego dotyczyło żądanie.
result	Wynik operacji, np. SUCCESS lub FAILURE.
details	Krótki opis zdarzenia bez zapisywania haseł i sekretów.
request_id	Unikalny identyfikator żądania, pomocny przy korelacji i analizie.

W projekcie uwzględniono zdarzenia LOGIN_SUCCESS, LOGIN_FAILED, LOGOUT, AUTHZ_DENIED, ACCESS_DENIED, ROLE_CHANGED oraz USER_CREATED. Rozróżnienie typów zdarzeń pozwala oddzielić poprawne działania użytkowników od nieudanych prób logowania i operacji zablokowanych przez kontrolę dostępu.

Identyfikator request_id jest generowany dla żądania i zapisywany wraz ze zdarzeniem. Ułatwia powiązanie wpisu w tabeli z konkretnym przebiegiem żądania. W starszych rekordach utworzonych przed wprowadzeniem migracji kolumny request_id pole może pozostać puste nie oznacza to błędu dla nowych zdarzeń.



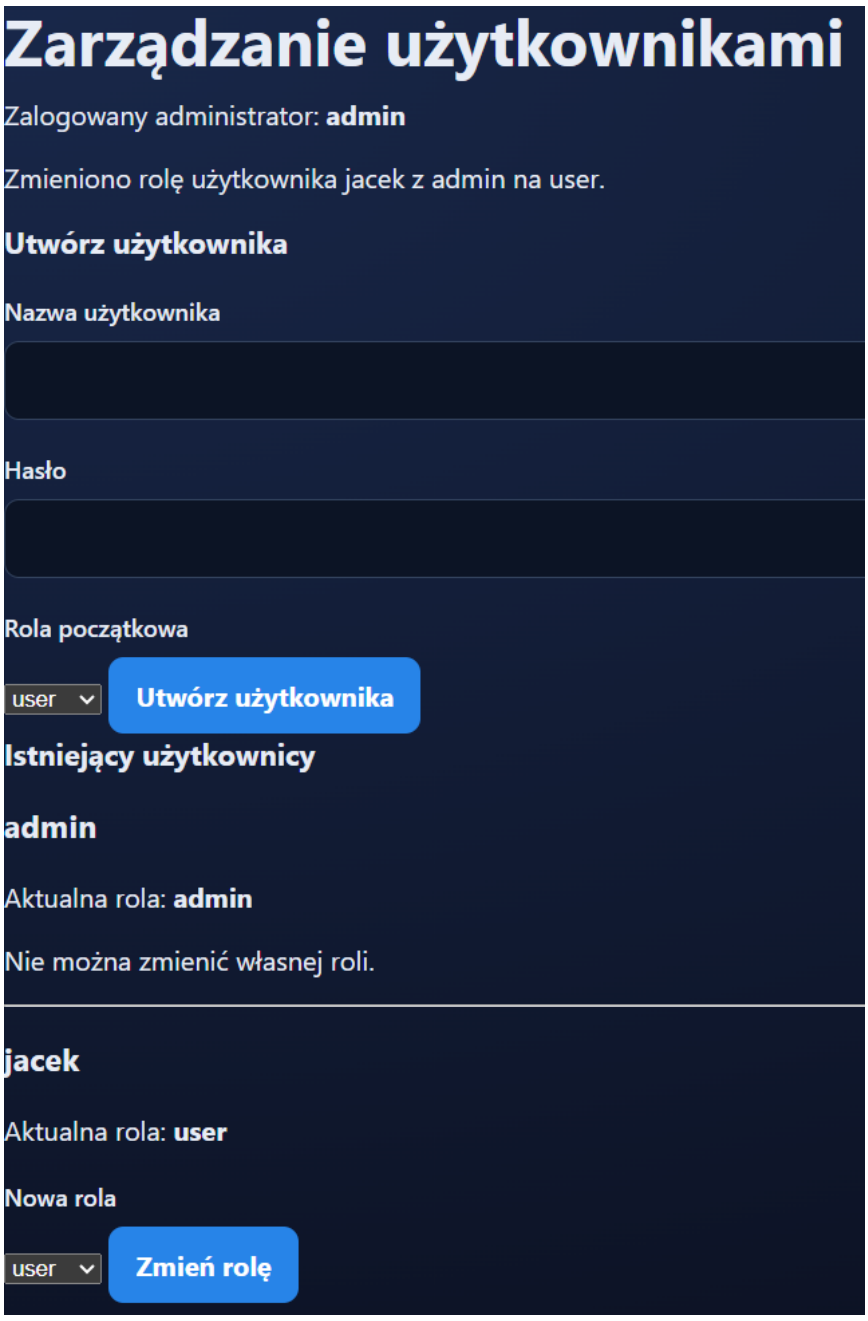
Rysunek 3. Przykładowe wpisy dziennika zdarzeń i identyfikatory korelacyjne.

Kontrola dostępu i zarządzanie rolami

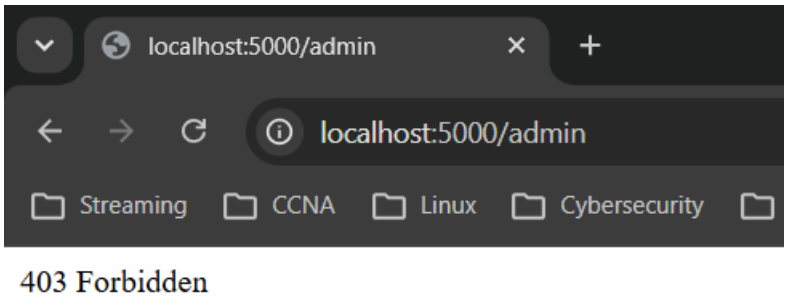
Aplikacja rozróżnia konto zwykłego użytkownika i administratora. Panel zarządzania kontami oraz rolami jest dostępny wyłącznie dla administratora. Użytkownik bez odpowiednich uprawnień nie powinien mieć możliwości wykonania operacji administracyjnej przez bezpośrednie wpisanie adresu URL.

Administrator może utworzyć konto i przypisać mu rolę, a także zmienić rolę istniejącego użytkownika. Hasła są przechowywane jako skróty tworzone przy użyciu mechanizmu haszowania haseł z biblioteki Werkzeug, a nie jako jawny tekst. Aplikacja sprawdza też uprawnienia po stronie serwera samo ukrycie linku w interfejsie nie jest traktowane jako zabezpieczenie.

Istotnym przypadkiem testowym jest odebranie roli administratora użytkownikowi, który ma już aktywną sesję. Aplikacja odświeża rolę sesji na podstawie aktualnego stanu w bazie danych. Po zmianie roli użytkownik nie powinien zachować dostępu administracyjnego wyłącznie dlatego, że zalogował się wcześniej.



Rysunek 3. Zmiana roli użytkownika przez administratora.



Rysunek 4. Odmowa dostępu po odebraniu uprawnień.



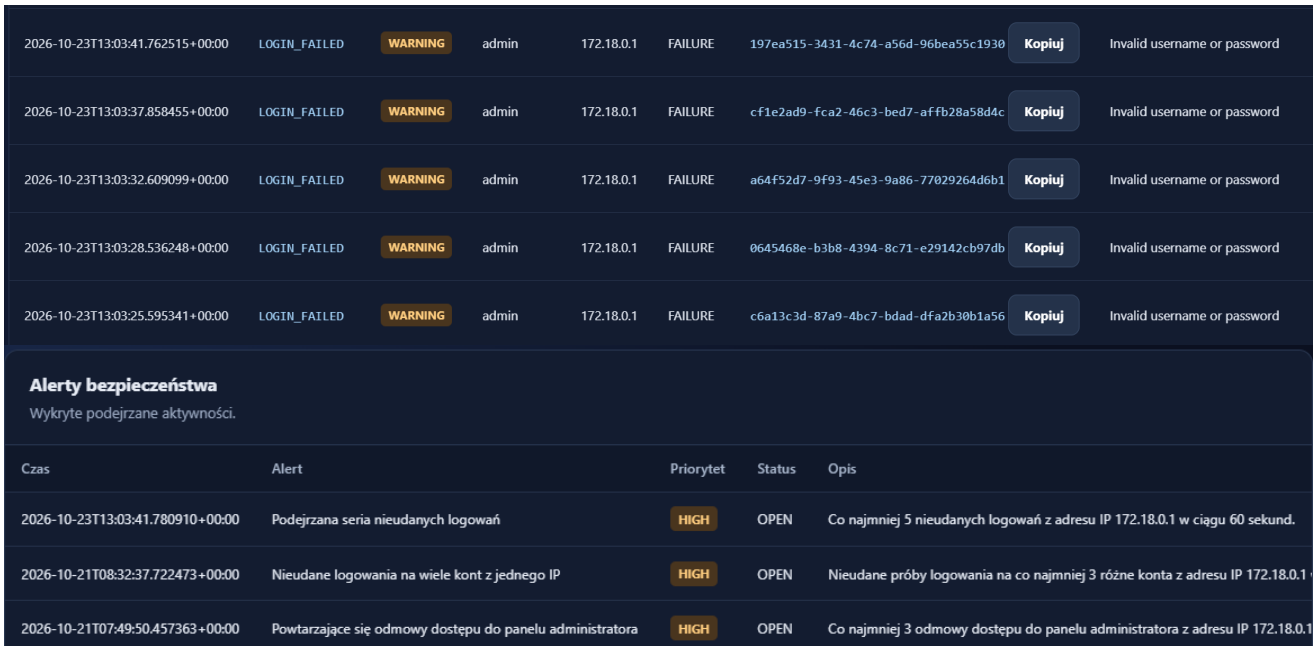
Rysunek 5. Rejestracja zmiany uprawnień i późniejszej odmowy dostępu.

Korelacja zdarzeń i alerty

Oprócz zapisywania pojedynczych zdarzeń aplikacja analizuje ich powtarzalność w krótkich oknach czasowych. Gdy liczba zdarzeń przekroczy ustalony próg, tworzony jest alert o wysokim poziomie ważności. Takie podejście pozwala zauważyć serię zdarzeń, która pojedynczo mogłaby nie wyglądać nietypowo.

Reguła	Warunek	Znaczenie
Seria nieudanych logowań	Co najmniej 5 nieudanych logowań z jednego IP w ciągu 60 sekund.	Możliwa próba odgadywania hasła.
Powtarzające się odmowy dostępu	Co najmniej 3 zdarzenia ACCESS_DENIED dla /admin z jednego IP w ciągu 60 sekund.	Powtarzające się próby wejścia do panelu administratora.
Logowania na wiele kont	Nieudane logowania na co najmniej 3 różne nazwy użytkowników z jednego IP w ciągu 60 sekund.	Możliwa próba sprawdzania wielu kont.

Alerty są wynikiem prostych reguł korelacyjnych, a nie automatycznym potwierdzeniem ataku. Administrator powinien sprawdzić kontekst zdarzeń i ustalić, czy aktywność była nieuprawniona. W środowisku produkcyjnym progi wymagałyby dostrojenia, aby ograniczyć liczbę fałszywych alarmów.

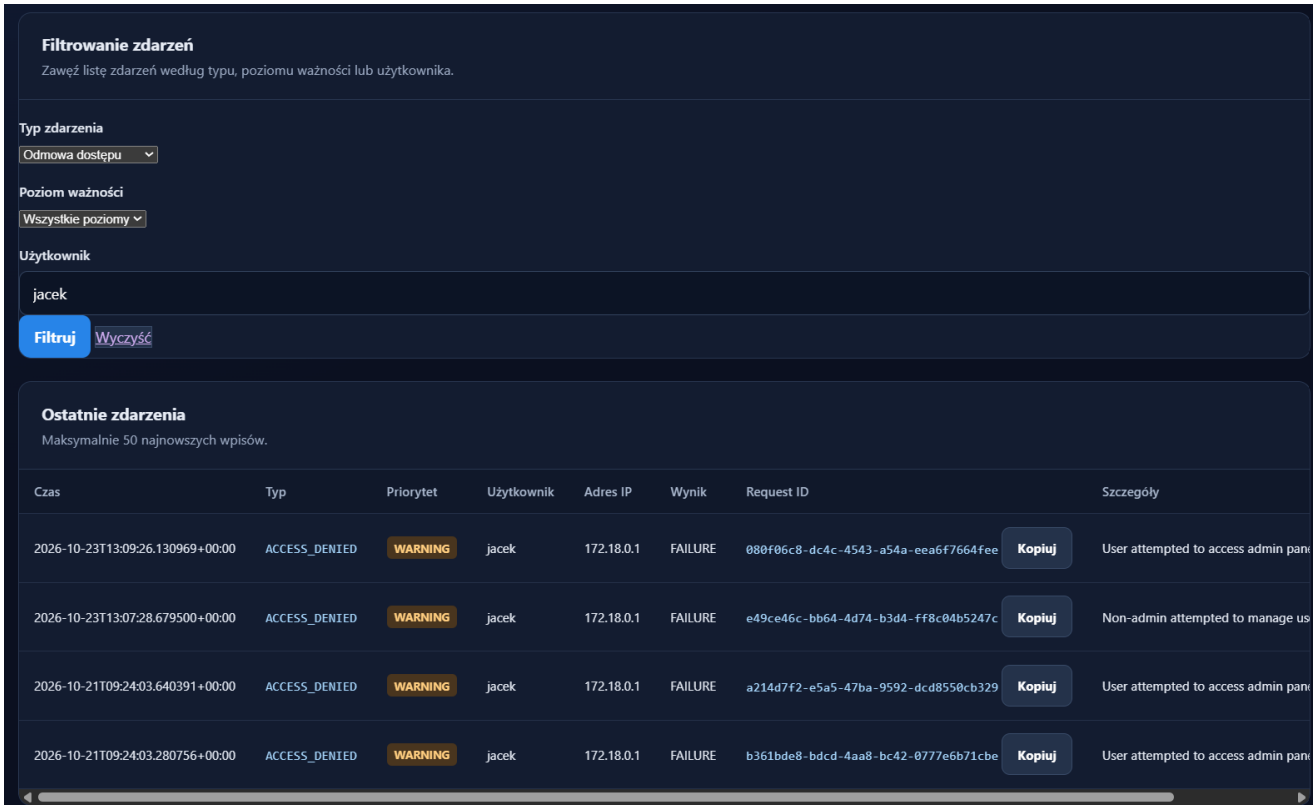


Rysunek 6. Alert wygenerowany po serii nieudanych logowań.

Scenariusze testowe i wyniki

Testy przeprowadzono ręcznie w lokalnym środowisku. Ich celem było sprawdzenie zarówno rejestrowania pojedynczych zdarzeń, jak i działania reguł korelacji oraz kontroli dostępu. Poniższa tabela podsumowuje scenariusze.

ID	Scenariusz	Oczekiwany rezultat	Wynik
T1	Poprawne logowanie administratora.	LOGIN_SUCCESS widoczny na dashboardzie dostęp do funkcji administracyjnych.	Zaliczone w testach ręcznych.
T2	Nieudane logowania na konto.	LOGIN_FAILED zapisany w bazie po przekroczeniu progu powstaje alert.	Zaliczone alert HIGH za serię prób.
T3	Użytkownik bez roli admin próbuje otworzyć panel administracyjny.	Żądanie odrzucone i zarejestrowane jako odmowa dostępu.	Zaliczone w testach ręcznych.
T4	Administrator zmienia rolę użytkownika.	Zapis ROLE_CHANGED nowa rola obowiązuje przy kolejnych żądaniach.	Zaliczone w testach ręcznych.
T5	Odebranie roli admin w aktywnej sesji i ponowna próba wejścia do /admin.	Odpowiedź 403 i zdarzenie ACCESS_DENIED użytkownik nie zachowuje uprawnień.	Zaliczone w teście retestowym.
T6	Powtarzające się odmowy dostępu do /admin.	Po 3 odmowach z tego samego IP w 60 sekund powstaje alert HIGH.	Reguła zaimplementowana i testowana.
T7	Nieudane logowania do wielu nazw użytkowników z jednego IP.	Alert HIGH po nieudanych logowaniach do co najmniej 3 kont w 60 sekund.	Reguła zaimplementowana i testowana.
T8	Przegląd zapisanych szczegółów zdarzeń.	Wpisy nie zawierają haseł ani sekretów mają kontekst zdarzenia i request_id dla nowych żądań.	Sprawdzić i udokumentować zrzutem.



Rysunek 7. Filtrowanie zdarzeń w dashboardzie.

Katalog zdarzeń bezpieczeństwa

Typ zdarzenia	Kiedy powstaje	Znaczenie / poziom
LOGIN_SUCCESS	Użytkownik poprawnie się uwierzytelnił.	Informacyjne poprawne logowanie.
LOGIN_FAILED	Podano nieprawidłowe dane logowania.	Ostrzegawcze może być elementem serii prób.
LOGOUT	Użytkownik wylogował się.	Informacyjne zakończenie sesji.
AUTHZ_DENIED	Operacja została odrzucona z powodu braku uprawnień.	Odmowa autoryzacji sprawdzić faktyczny poziom w kodzie.
ACCESS_DENIED	Odmowa dostępu do chronionego zasobu, w testach m.in. panelu /admin.	Ostrzegawcze powtarzalność może generować alert.
ROLE_CHANGED	Administrator zmienił rolę istniejącego konta.	Zdarzenie audytowe zmiana uprawnień.
USER_CREATED	Administrator utworzył nowe konto.	Zdarzenie audytowe utworzenie tożsamości.

Bezpieczne logowanie i ochrona danych

Dzienniki bezpieczeństwa powinny być użyteczne do analizy, ale nie mogą same stać się źródłem wycieku. W projekcie szczegóły zdarzeń mają opisywać przebieg operacji, a nie przechowywać haseł lub kluczy konfiguracyjnych. Hasła użytkowników są haszowane, a plik .env wyłączono z kontroli wersji.

Ochrona danych w dziennikach została oceniona w ograniczonym zakresie manualnym. Projekt nie implementuje pełnego systemu retencji, kontroli integralności logów, centralnego magazynu, ani szczegółowego modelu uprawnień do odczytu logów.

Ocena ryzyka

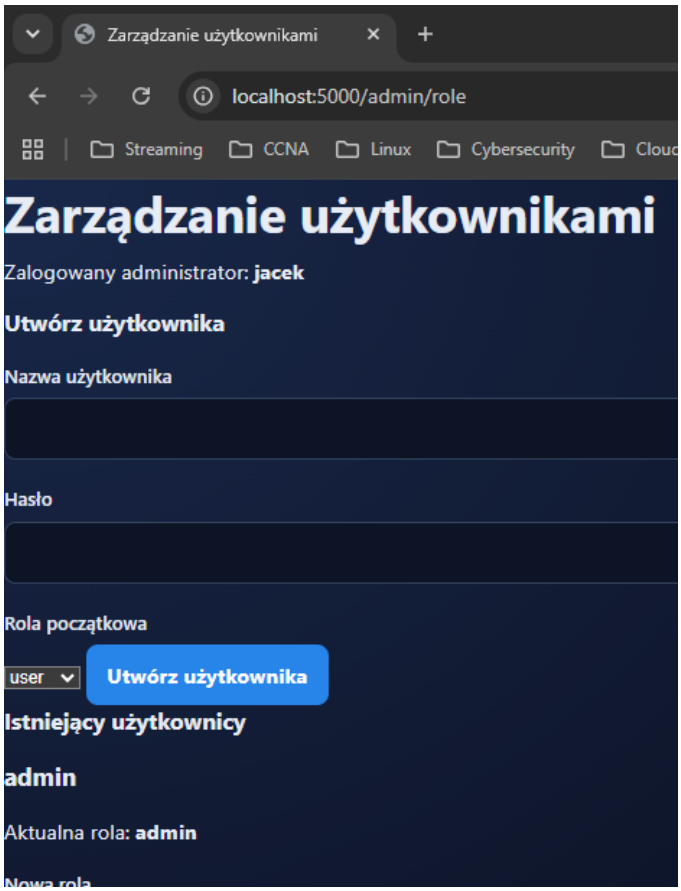
Poniższa ocena jest jakościową oceną ryzyka dla demonstracyjnej aplikacji lokalnej. Skala prawdopodobieństwa i wpływu jest uproszczona: L jako niskie, M jako średnie, W dla wysokie. Priorytet wynika z połączenia prawdopodobieństwa i wpływu, a nie z formalnej analizy ilościowej.

Ryzyko	Prawdopodobieństwo	Wpływ	Priorytet	Ograniczenie ryzyka
Seria prób logowania nie zostanie zauważona.	M	H	Wysoki	Rejestrowanie LOGIN_FAILED i reguła alertu dla 5 prób w 60 sekund sprawdzić alert w teście.
Hasło lub sekret trafi do dziennika.	M	H	Wysoki	Nie zapisywać danych uwierzytelniających w details wykonać kontrolę logów po testowym błędnym logowaniu.
Użytkownik zachowa uprawnienia po zmianie roli.	M	H	Wysoki	Odświeżać rolę z bazy przy kolejnych żądaniach i wykonać retest aktywnej sesji.
Logi zostaną zmodyfikowane lub usunięte przez osobę z dostępem do hosta/bazy.	L/M	H	Średni	Ograniczyć dostęp do hosta i pliku bazy w systemie produkcyjnym wysyłać logi do odrębnego, kontrolowanego magazynu.
Wstrzyknięcie znaków sterujących do pola details lub username zniekształci wpisy.	M	M	Średni	Walidować lub neutralizować znaki sterujące i formatować logi strukturalnie nie uznawać tego za przetestowane bez testu.
Błędne zaufanie do X-Forwarded-For za proxy spowoduje zapis fałszywego IP.	M	M	Średni	Nie ufać nagłówkowi od dowolnego klienta konfigurować zaufane proxy i weryfikować źródło IP.

Najwyższy priorytet przypisano ryzykom dotyczącym niewykrycia prób logowania, ujawnienia sekretów w logach i utrzymania nieaktualnych uprawnień, ponieważ mogą one osłabić wykrywanie incydentów albo doprowadzić do nieuprawnionego dostępu. Ryzyka log injection i błędnego zaufania do X-Forwarded-For opisano jako ograniczenia do dalszej weryfikacji, a nie jako podatności potwierdzone testami.

Porównanie BEFORE - AFTER

BEFORE: Przed poprawką należało zweryfikować, czy po odebraniu roli administratora użytkownik nadal mógł uzyskać dostęp do panelu administracyjnego. Kod sprawdzał przypisanie roli per sesja, czyli jeżeli użytkownik miał przypisaną role admina przy rozpoczeciu sesji, nawet przy zmianie jej na zwykłego użytkownika miał dostęp do panelu administratora.



AFTER: Dodano odświeżanie roli użytkownika na podstawie danych z bazy. Po zmianie roli konta `jacek` z `admin` na `user` próba wejścia na `/admin` zakończyła się odpowiedzią `403 Forbidden`. Na dashboardzie pojawiły się zdarzenia `ROLE_CHANGED` i `ACCESS_DENIED`.

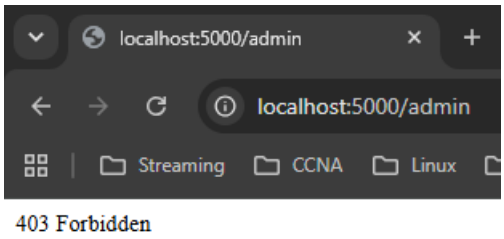
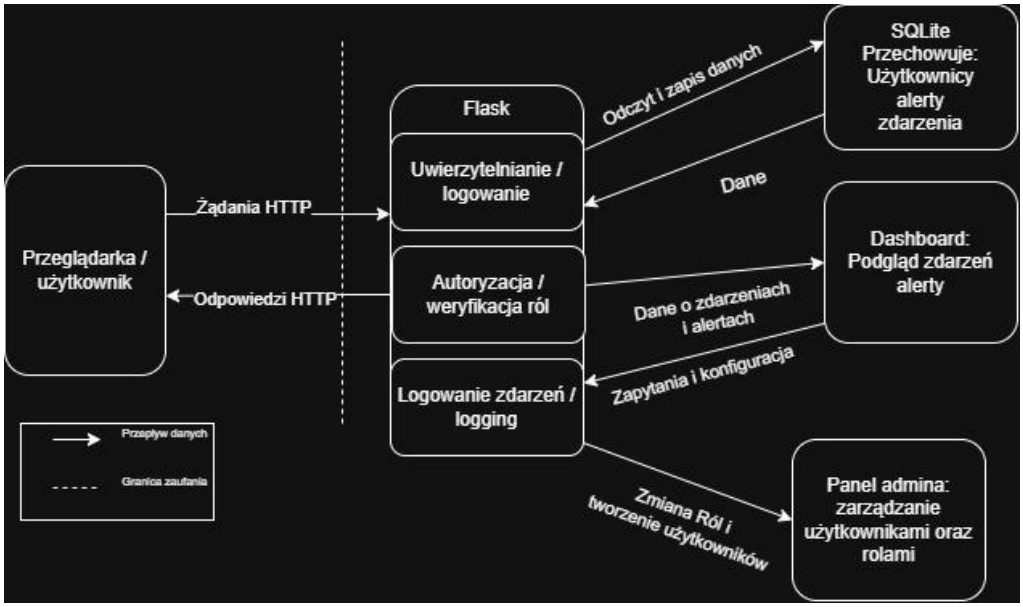


Diagram przepływu danych (DFD)

Diagram przedstawia przepływ danych w aplikacji monitorowania bezpieczeństwa. Użytkownik komunikuje się z aplikacją Flask za pośrednictwem przeglądarki. Flask odpowiada za uwierzytelnianie, weryfikację uprawnień i rejestrowanie zdarzeń. Dane użytkowników, zdarzenia i alerty są przechowywane w bazie SQLite i prezentowane na dashboardzie administratora. Zaznaczono również granicę zaufania między przeglądarką a aplikacją.



Odniesienie do standardów

Projekt odnosi się do OWASP Top 10:2025, w szczególności kategorii A09 „Logging & Alerting Failures". Kategoria ta jest bezpośrednio związana z celem projektu: rejestrowaniem zdarzeń bezpieczeństwa, wykrywaniem wzorców oraz prezentowaniem alertów. Samo zapisanie wpisu nie wystarcza, jeżeli nikt nie może go przeanalizować albo nie ma mechanizmu zauważenia powtarzających się niepowodzeń.

Drugim punktem odniesienia jest OWASP Application Security Verification Standard (ASVS) 5.0.0, wykorzystywany w projekcie jako rama do myślenia o weryfikowalnych wymaganiach bezpieczeństwa aplikacji. W ramach demonstracji sprawdzono praktycznie rejestrowanie zdarzeń, kontrolę dostępu i zachowanie aplikacji po zmianie uprawnień. Weryfikacja ma charakter testów funkcjonalnych i bezpieczeństwa w ograniczonym zakresie. Nie należy interpretować jej jako pełnej certyfikacji zgodności z OWASP Top 10 lub ASVS.

Mapowanie testów do źródeł

Obszar / test	Źródło	Związek z projektem	Status
Rejestrowanie zdarzeń i alarmowanie	OWASP Top 10:2025, A09 — Logging & Alerting Failures	Zdarzenia logowania, odmowy dostępu, alerty korelacyjne i dashboard.	Zaimplementowane dowody w raporcie.
Weryfikacja wymagań bezpieczeństwa aplikacji	OWASP ASVS 5.0.0	Rama do dalszego przypisania testów do konkretnych wymagań dotyczących logowania, uwierzytelniania i kontroli dostępu.	Mapowanie numerów wymaga weryfikacji.

Testowanie scenariuszy webowych	OWASP Web Security Testing Guide 4.2	Ręczne testy logowania, kontroli dostępu i zachowania aplikacji.	Odniesienie metodyczne nie oznacza pełnego wykonania WSTG.

Ograniczenia i możliwe usprawnienia

Najważniejszym ograniczeniem jest lokalny, demonstracyjny charakter środowiska. Reguły korelacyjne są oparte na stałych progach, a alerty nie zawierają pełnego procesu obsługi incydentu, potwierdzania fałszywych alarmów ani eskalacji. SQLite sprawdza się w małym projekcie, lecz nie zastępuje centralnej platformy logowania dla większej liczby usług.

W kolejnych wersjach można rozważyć wysyłanie zdarzeń do centralnego systemu monitorowania, konfigurowalne progi alertów, testy automatyczne, kontrolę retencji, ochronę integralności logów, eksport raportów oraz bardziej szczegółowe mapowanie scenariuszy do ASVS. W środowisku produkcyjnym należałoby także przeprowadzić analizę wydajności, testy odporności na zalewanie logami i przegląd uprawnień do samego dashboardu.

Wnioski

Projekt pokazuje podstawowy przepływ obserwowalności bezpieczeństwa: aplikacja zapisuje zdarzenie, przypisuje mu kontekst, analizuje powtarzalne zachowania i udostępnia administratorowi wynik w panelu. Dzięki typom zdarzeń, poziomom ważności, filtrom i identyfikatorowi request_id łatwiej ustalić, kto wykonał operację, kiedy do niej doszło i czego dotyczyła.

Testy potwierdziły działanie najważniejszych funkcji demonstracyjnych: logowania i rejestrowania zdarzeń, alertu po serii nieudanych logowań, odmowy dostępu do panelu administracyjnego oraz odebrania uprawnień aktywnej sesji. Rozwiązanie stanowi punkt wyjścia do dalszych prac, ale nie jest pełnym systemem SIEM ani produkcyjnym rozwiązaniem monitorowania.

Najważniejszym rezultatem jest połączenie rejestrowania zdarzeń z ich korelacją i prezentacją. Taki układ zwiększa możliwość zauważenia podejrzanej aktywności w porównaniu z samym przechowywaniem nieustrukturyzowanych komunikatów.

Sposób odtworzenia demonstracji

1. Uruchomić Docker Desktop.

2. W katalogu projektu sprawdzić, czy plik .env istnieje i zawiera lokalnie wygenerowany SECRET_KEY. Nie publikować jego treści.

3. W PowerShell wykonać docker compose up -d --build, a następnie docker compose ps.

4. Otworzyć http://localhost:5000 i zalogować się kontem testowym.

5. Wykonać serię nieudanych logowań, po czym sprawdzić alert na dashboardzie.

6. W osobnej sesji użytkownika testowego sprawdzić odmowę dostępu, a następnie jako administrator zmienić jego rolę i powtórzyć próbę w tej samej sesji.

7. Sprawdzić, czy dashboard pokazuje ROLE_CHANGED i ACCESS_DENIED oraz czy szczegóły zdarzeń nie zawierają haseł ani sekretów.

8. Po zakończeniu demonstracji zatrzymać środowisko poleceniem docker compose down, jeśli nie jest już potrzebne.